مكتب

Reference Notes

Week 9: List Algorithms

7th Grade Computer Science - Module 2

1 Introduction

1.1 This Week's Big Question

How can we systematically search through and organize data in our lists? When we have hundreds or thousands of items in a list, how do we find what we need quickly? Why do some approaches work better than others?

Prerequisites

Before starting this week, you should be comfortable with:

- Creating and modifying lists (Week 7)
- Understanding list mutability and references (Week 8)
- Using loops to iterate through lists
- Working with list indices

1.2 What You Already Know

You've learned how to create lists, add and remove items, and understand how lists behave in memory. You know that lists are mutable and can track how changes affect them. You can iterate through lists using for loops and access items by their index positions.

1.3 What You'll Be Able to Do

By the end of this week, you'll be able to search through lists to find specific items, sort lists to organize your data, recognize common patterns in list problems, and even create new lists using the powerful list comprehension syntax. You'll have the tools to handle real-world data processing tasks!

2 VIDEO 1: List Algorithms - Patterns for Success

2.1 Understanding List Algorithms

An algorithm is simply a step-by-step process for solving a problem. When we work with lists, certain patterns appear again and again - finding items, sorting data, filtering results. These patterns are so common that programmers have developed standard approaches that work reliably every time.

2.2 Common Algorithm Patterns

Let's explore the three most common patterns you'll use with lists. Notice how each one follows a clear structure that you can adapt to many different problems:

```
# Pattern 1: Accumulator (collecting results)
total = 0
for num in numbers:
    total = total + num

# Pattern 2: Filter (keeping certain items)
evens = []
for num in numbers:
    if num % 2 == 0:
        evens.append(num)
```

See how both patterns start with an empty result and build it up step by step? This is the foundation of most list algorithms.

2.3 Building More Complex Patterns

Once you understand basic patterns, you can combine them to solve harder problems. Here's how we might find the largest even number in a list:

```
# Combining filter and max-finding patterns
largest_even = None
for num in numbers:
    if num % 2 == 0:  # Filter pattern
        if largest_even is None or num > largest_even:
            largest_even = num  # Max pattern
print(f"Largest even: {largest_even}")
```

This combines filtering (checking if even) with tracking a maximum value - two patterns working together!

Tip

When solving a new list problem, ask yourself: "Which pattern does this remind me of?" Most problems are variations of accumulate, filter, or search patterns.

Quick Check

Given the list [4, 7, 2, 9, 3], trace through this algorithm:

```
count = 0
for num in numbers:
    if num > 5:
        count = count + 1
```

What will count equal at the end?

3 VIDEO 2: Linear Search - Finding What You Need

3.1 Understanding Linear Search

Linear search is the simplest way to find something in a list: look at each item one by one until you find what you're looking for. It's like looking for your favorite book on a shelf by checking each book from left to right.

3.2 Basic Linear Search

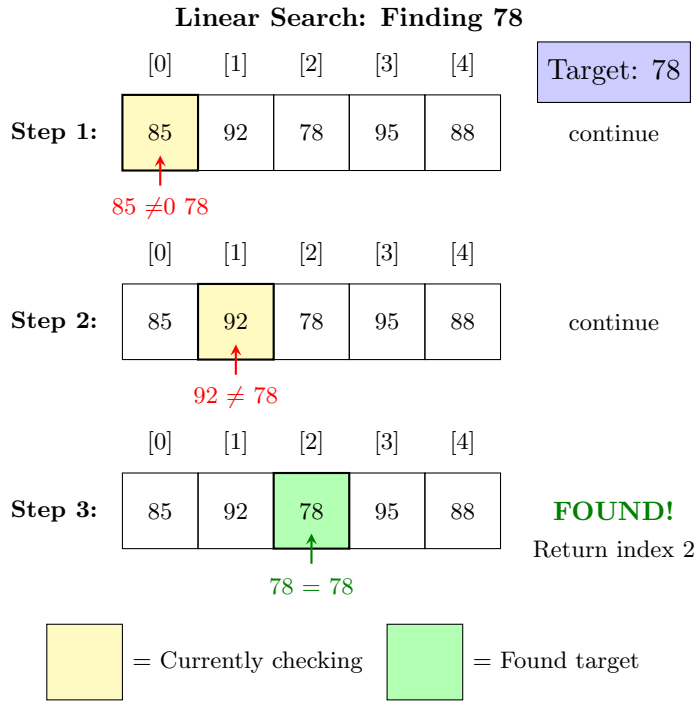
Here's the classic linear search algorithm. Notice how it returns immediately when it finds the target - we don't need to check the rest of the list:

```
def find_item(my_list, target):
    for i in range(len(my_list)):
        if my_list[i] == target:
            return i  # Found it! Return the index
    return -1  # Not found

scores = [85, 92, 78, 95, 88]
position = find_item(scores, 78)
print(f"Found at index: {position}")  # Output: 2
```

The function returns the index where the item was found, or -1 if it wasn't in the list at all.

3.3 Visualizing the Search Process

Let's see how linear search examines each element until it finds the target or reaches the end:



The search stops as soon as we find the target - we don't waste time checking the remaining items.

3.4 Search Variations

Linear search can be adapted for different needs. Here are two useful variations you'll often need:

```
# Finding ALL occurrences
def find_all(my_list, target):
    positions = []
    for i in range(len(my_list)):
        if my_list[i] == target:
            positions.append(i)
    return positions

# Finding with a condition
def find_first_passing(grades):
    for grade in grades:
        if grade >= 60:
            return grade
    return None
```

These variations show how flexible the linear search pattern is - you can adapt it to many situations!

Common Error

Common Mistake: Forgetting to handle the "not found" case. Always decide what to return when the item isn't in the list (common choices: -1, None, or an empty list).

Quick Check

Write a linear search that finds the LAST occurrence of an item in a list. Hint: Search backwards!

4 VIDEO 3: Basic Sorting - Organizing Your Data

4.1 Understanding Sorting

Sorting puts items in order - alphabetical, numerical, or by any rule you define. Python makes sorting easy with the built-in sort() method and sorted() function. Understanding when to use each one is key to writing efficient programs.

4.2 Sort vs Sorted

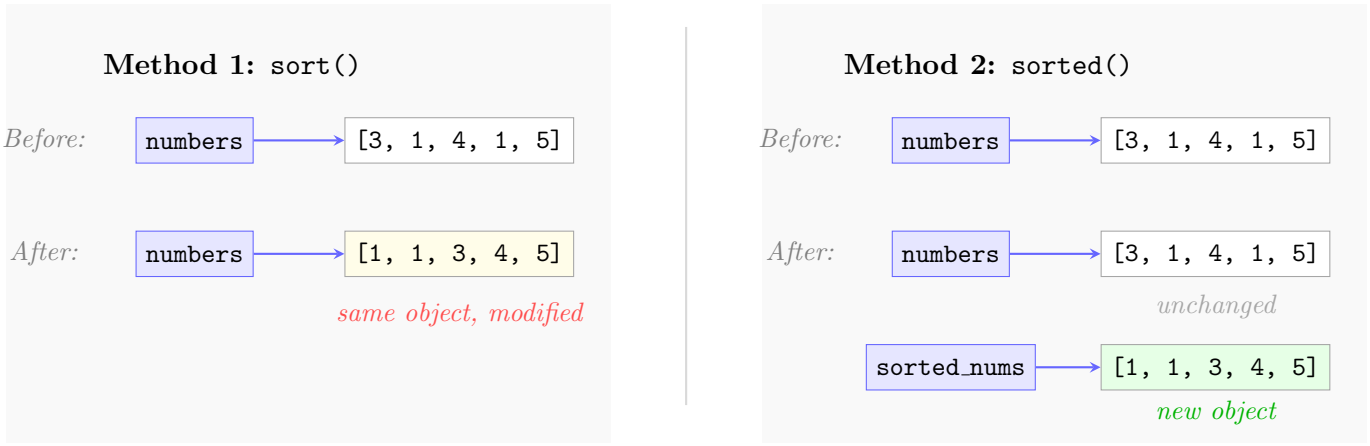
Python gives us two ways to sort, and they behave very differently. Let's see both in action:

```
# Method 1: sort() - modifies the original list
numbers = [3, 1, 4, 1, 5]
numbers.sort()  # Changes numbers directly
print(numbers)  # [1, 1, 3, 4, 5]

# Method 2: sorted() - creates a new sorted list
numbers = [3, 1, 4, 1, 5]
sorted_nums = sorted(numbers)  # Original unchanged
print(numbers)       # Still [3, 1, 4, 1, 5]
print(sorted_nums)   # [1, 1, 3, 4, 5]
```

See the difference? sort() changes your list permanently, while sorted() gives you a new sorted list and leaves the original alone.

4.3 Visualizing Sort vs Sorted

Understanding the memory difference between sort() and sorted() helps you choose the right one:



The key insight: sort() is more memory-efficient but changes your data. Choose sorted() when you need to keep the original order.

4.4 Custom Sorting Rules

Sometimes you need to sort by special rules. Python lets you customize how sorting works:

```
# Sort by different criteria
words = ["cat", "elephant", "dog", "ant"]
words.sort(key=len)  # Sort by length
print(words)  # ['cat', 'dog', 'ant', 'elephant']

# Reverse sorting
scores = [78, 92, 65, 88, 95]
scores.sort(reverse=True)  # Highest first
print(scores)  # [95, 92, 88, 78, 65]
```

The key parameter lets you tell Python what to sort by - length, first letter, or any rule you can imagine!

Rule

Use list.sort() when you want to modify the list permanently. Use sorted(list) when you need to preserve the original order. Both are equally correct - choose based on your needs!

Quick Check

You have grades = [82, 90, 78]. If you run sorted_grades = sorted(grades), then grades.sort(), what will grades and sorted_grades contain?

5 VIDEO 4: List Comprehensions - Python's Power Tool

5.1 Understanding List Comprehensions

List comprehensions are Python's elegant way to create new lists from existing ones. They combine a loop and optional filter into a single, readable line. Think of them as a recipe: "give me all items

that match this pattern."

5.2 Basic List Comprehension

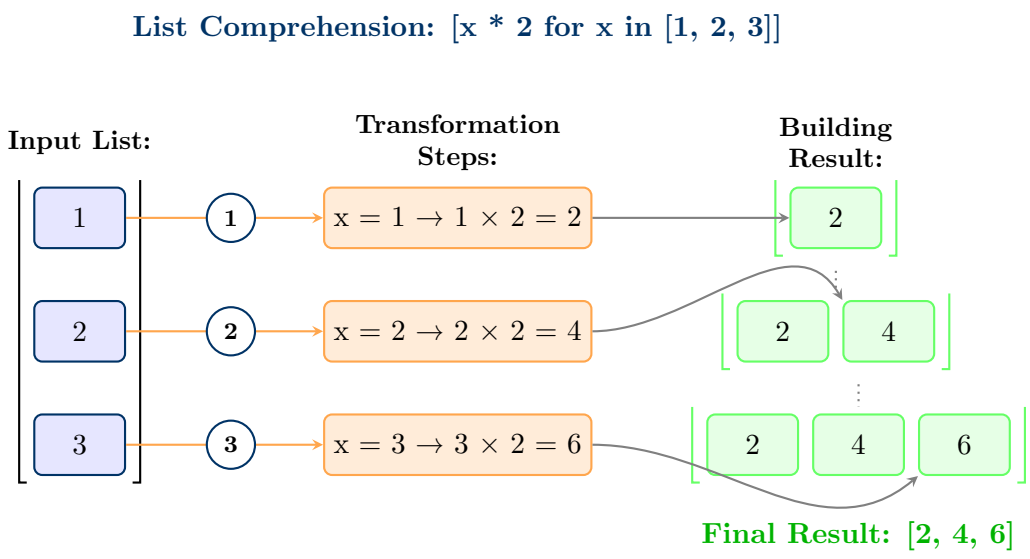
Let's see how list comprehensions replace traditional loops. Watch how much cleaner the code becomes:

```
# Traditional way with a loop
squares = []
for x in range(5):
    squares.append(x ** 2)

# List comprehension way - same result!
squares = [x ** 2 for x in range(5)]
print(squares)  # [0, 1, 4, 9, 16]

# With strings
words = ["hello", "world", "python"]
lengths = [len(word) for word in words]
print(lengths)  # [5, 5, 6]
```

Notice the pattern: [expression for item in list]. It reads almost like English: "give me x squared for each x in range(5)".

5.3 Visualizing Comprehension Flow

Let's trace through how Python processes a list comprehension step by step:



Each item flows through the expression and the results build up into a new list - all in one line!

5.4 Adding Filters with If

List comprehensions become even more powerful when you add conditions. Now you can filter and transform in one step:

```
# Get only even numbers, doubled
numbers = [1, 2, 3, 4, 5, 6]
evens_doubled = [x * 2 for x in numbers if x % 2 == 0]
print(evens_doubled)  # [4, 8, 12]
```

```
# Get passing grades only
all_grades = [45, 82, 67, 90, 55, 78]
passing = [g for g in all_grades if g >= 60]
print(passing)  # [82, 67, 90, 78]
```

The if clause acts like a gatekeeper - only items that pass the test make it into the new list.

Tip

List comprehensions are perfect for simple transformations and filters. If your logic needs multiple lines or is complex, stick with a regular for loop for clarity.

Quick Check

Convert this loop to a list comprehension:

```
result = []
for n in [1, 2, 3, 4, 5]:
    if n > 2:
        result.append(n * 10)
```

6 VIDEO 5: Avoiding Common Pitfalls

6.1 The Iteration Modification Trap

One of the trickiest bugs happens when you try to remove items from a list while looping through it. Python gets confused about where it is in the list!

6.2 Why Modification During Iteration Fails

Here's what happens when you try to remove items while iterating - the results might surprise you:

```
# WRONG - Don't do this!
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    if num % 2 == 0:
        numbers.remove(num)
print(numbers)  # [1, 3, 5] - Wait, where's 4?

# The problem: removing shifts indices!
# When we remove 2, everything shifts left
# Python skips checking 3!
```

When you remove an item, all later items shift position, but Python doesn't adjust its position in the loop. Items get skipped!

6.3 Safe Ways to Modify Lists

There are several safe patterns for modifying lists. Choose the one that fits your situation:

```
# Method 1: Create a new list (cleanest)
numbers = [1, 2, 3, 4, 5]
odds = [n for n in numbers if n % 2 != 0]

```

```
# Method 2: Iterate over a copy
for num in numbers.copy():  # or numbers[:]
    if num % 2 == 0:
        numbers.remove(num)

# Method 3: Iterate backwards (for removal)
for i in range(len(numbers)-1, -1, -1):
    if numbers[i] % 2 == 0:
        numbers.pop(i)
```

Each method has its use: new lists are clearest, copying works for small lists, and backwards iteration is efficient for large lists.

Common Error

Common Mistake: Modifying a list while iterating through it. This causes items to be skipped or errors. Always use one of the safe patterns: create a new list, iterate over a copy, or iterate backwards when removing items.

Quick Check

What will this print?

```
items = [1, 2, 3]
for item in items:
    items.append(item * 2)
```

(Careful - this creates an infinite loop!)

7 VIDEO 6: Summary

7.1 What We Learned This Week

This week, you mastered the essential algorithms for working with lists! You learned to recognize common patterns like accumulator and filter that appear in many problems. You can now search through lists systematically with linear search, organize data using Python's powerful sorting tools, create lists efficiently with comprehensions, and avoid the tricky pitfall of modifying lists during iteration. These skills form the foundation for handling real-world data processing tasks!

Landmark Moment

You now understand list algorithms - the building blocks for data processing! From searching student records to sorting high scores to filtering results, you have the tools to handle collections of data like a professional programmer. These patterns appear everywhere in programming!

8 Quick Reference

Quick Reference

Week 9 Essential Patterns:
Algorithm Patterns:

- Pattern: `accumulator = 0; for item in list: accumulator += item`
- Common use: Sums, counts, collecting results
- Common mistake: Forgetting to initialize accumulator

Linear Search:

- Pattern: `for i, item in enumerate(list): if condition: return i`
- Common use: Finding positions, checking existence
- Debug tip: Always handle the "not found" case

Sorting:

- Pattern: `list.sort()` or `new_list = sorted(list)`
- Memory: sort() modifies in-place, sorted() creates new list
- Common mistake: Using sort() when you need the original order

List Comprehensions:

- Pattern: `[expression for item in list if condition]`
- Common use: Transform and/or filter in one line
- Debug tip: If it's getting complex, use a regular loop instead

Safe List Modification:

- Pattern: `new_list = [item for item in old if keep(item)]`
- Common mistake: Removing items while iterating forward
- Debug tip: Create new list or iterate over a copy

8.1 Reflection Questions

1. Which algorithm pattern (accumulator, filter, search) did you find most challenging, and why?
2. How did visualizing the search and sort processes help you understand them better?
3. When debugging list algorithms, what strategy helped you most - print statements, tracing on paper, or something else?
4. Can you think of a real app or game where you'd need to search and sort lists?
5. How do list comprehensions change the way you think about creating new lists?

Next week: We'll explore 2D lists and matrix operations - imagine lists inside lists! You'll build grid-based games and learn to work with table-like data structures.